

RK3562 自定义启动 LOGO



主题	RK3562 自定义启动 LOGO
文档号	1.0
创建时间	2025-04-18
最后修改	2025-04-18
版本号	1.0
文件名	RK3562 自定义启动 LOGOV1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-04-18	张欢	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 制作开机启动 LOGO	5
1.1 开机 LOGO 格式规范	5
1.2 制作开机 LOGO 流程	5
第 2 章 替换开机 LOGO	8
2.1 瑞芯微平台开机 LOGO 配置要求	8
2.2 在 SDK 中替换和验证开机 LOGO	8
2.2.1 准备 LOGO 图片	8
2.2.2 上传 LOGO 图片到 SDK	8
2.2.3 重新编译内核	9
2.2.4 烧录并验证 LOGO	9

第 1 章 制作开机启动 LOGO

1.1 开机 LOGO 格式规范

根据瑞芯微平台的开机 LOGO 要求，LOGO 图片需满足以下格式规范：

- 文件格式：BMP 格式，24 位色深位图（即每个像素使用 24 位表示颜色）。
- 分辨率限制：LOGO 图片的分辨率应不超过目标显示设备的最大分辨率。为了确保兼容性和最佳显示效果，建议适配标准 HDMI 显示器的分辨率（例如 1920x1080）。

1.2 制作开机 LOGO 流程

下面将介绍如何在 Windows 11 系统上使用系统自带的“画图”工具制作开机启动 LOGO。具体步骤如下：

1) 准备源文件

首先，选择需要转换的原始图片文件（支持 JPG、PNG 等常见格式），这里以本公司 LOGO “合众恒跃”图片为例进行说明。使用 Windows 11 系统中的“画图”工具打开该图片。如图 1.2-1 所示。

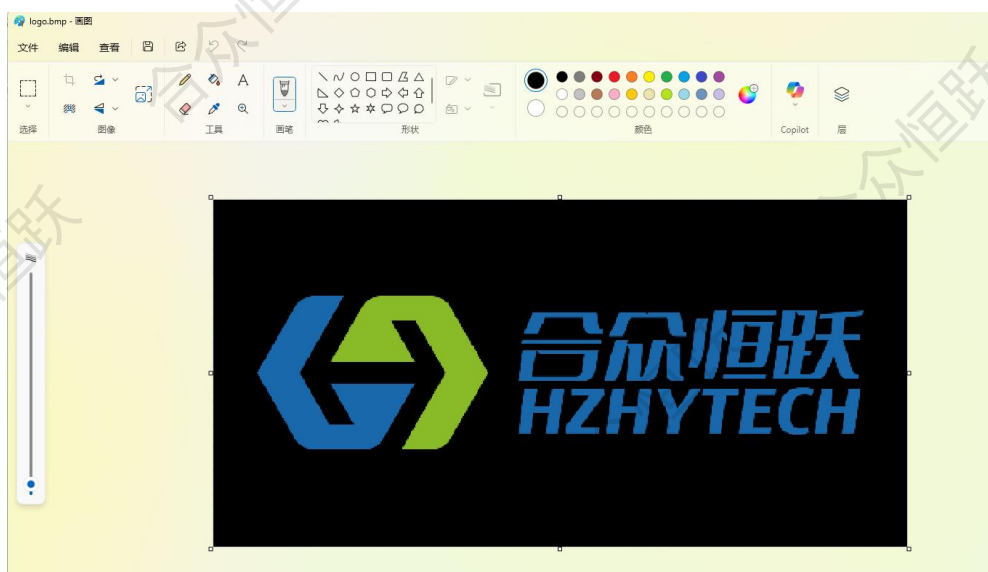


图 1.2-1 打开待转换图片

2) 分辨率调整

打开 LOGO 图片后，在“画图”工具界面，点击位于“主页”选项卡下的“调整大小”功能。在弹出的像素设置界面中，输入不超过显示器分辨率的数值。在此示例中，将 LOGO 图片水平分辨率设置为 540，垂直分辨率设置为 270。如图 1.2-2 所示。

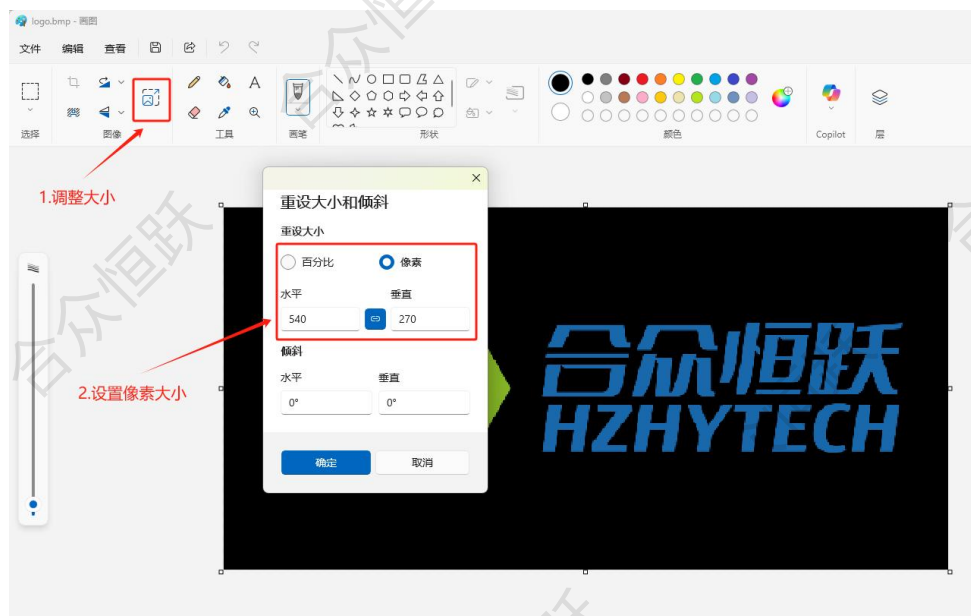


图 1.2-2 调整分辨率

3) 格式转换

选择“文件”菜单，然后点击“另存为”，选择“BMP 图片”，如图 1.2-3 所示。

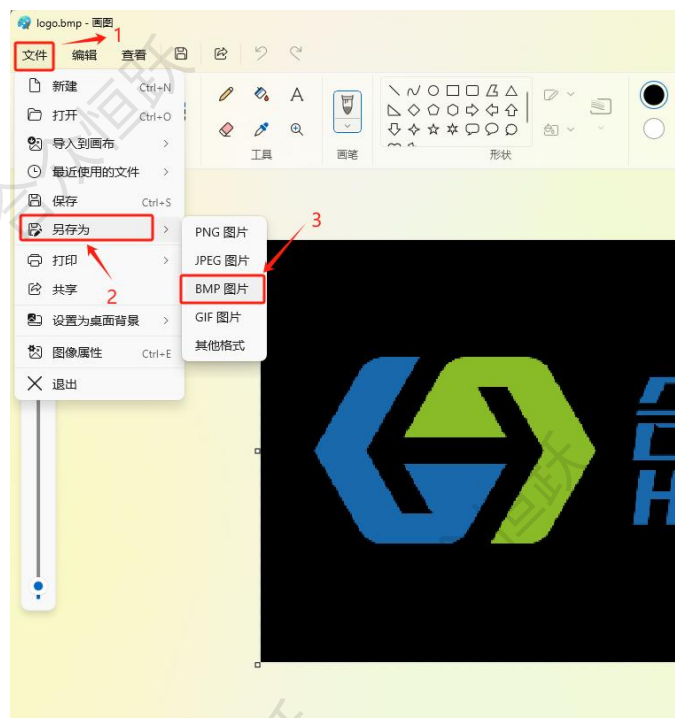


图 1.2-3 另存图片

在弹出的对话框中，保存类型设置为“24 位位图 (*.bmp;*.dib)”，如图 1.2-4 所示。

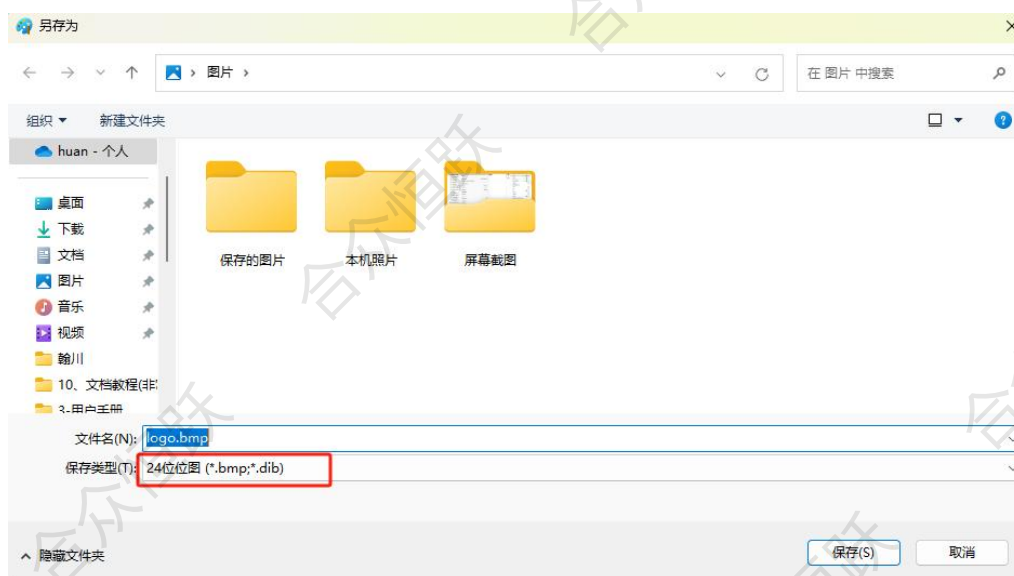


图 1.2-4 设置格式

第 2 章 替换开机 LOGO

2.1 瑞芯微平台开机 LOGO 配置要求

瑞芯微平台的开机 LOGO 包括两个部分：U-Boot 下的 LOGO 和 Kernel 下的 LOGO。两个 LOGO 图片都必须满足以下格式要求：

- 文件格式：BMP（24 位位图）
- 命名规则：

U-Boot 下的 LOGO 图片必须命名为 logo.bmp

Kernel 下的 LOGO 图片必须命名为 logo_kernel.bmp

- 准备 LOGO 图片：

U-Boot 下的 LOGO 图片应命名为 logo.bmp

Kernel 下的 LOGO 图片应命名为 logo_kernel.bmp

- 打包进 boot.img：

最终 uboot 和 kernel 的 LOGO 图片会被打包进 boot.img 中。

2.2 在 SDK 中替换和验证开机 LOGO

2.2.1 准备 LOGO 图片

首先将第 1 章制作好的 LOGO 图片重命名为 logo.bmp 和 logo_kernel.bmp，如果用户需要 uboot 和 kernel 下有不同的启动 LOGO 可以参考第 1 章节内容制作不同的启动 LOGO 图片，并分别命名为 logo.bmp 和 logo_kernel.bmp。

2.2.2 上传 LOGO 图片到 SDK

将重命名后的 LOGO 图片上传到 SDK 的 kernel 源码目录中（SDK 在购买我司对应的产品后会一同开源给用户）。

上传完成后，LOGO 图片在 SDK 的 kernel 目录中的位置如图 2.2-1 所示。

```
hzh@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ cd kernel
hzh@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel$ ls
android
arch
block
boot.img
boot.its
build.config.aarch64
build.config.allmodconfig
build.config.allmodconfig.aarch64
build.config.allmodconfig.arm
build.config.allmodconfig.x86_64
build.config.amlogic
build.config.arm
build.config.common
build.config.db845c
build.config.gki
build.config.gki.aarch64
build.config.gki.aarch64.fips140
build.config.gki.aarch64.fips140_eval_testing
build.config.gki-debug.aarch64
build.config.gki-debug.x86_64
build.config.gki.kasan
build.config.gki.kasan.aarch64
build.config.gki.kasan.x86_64
build.config.gki.kprobes
build.config.gki.kprobes.aarch64
build.config.gki.kprobes.x86_64
build.config.gki.x86_64
build.config.hikey960
build.config.khwasan
build.config.rockchip
build.config.x86_64
certs
COPYING
CREDITS
crypto
Documentation
drivers
fs
include
init
io_uring
ipc
Kbuild
Kconfig
kernel
lib
LICENSES
logo.bmp
logo_kernel.bmp
MAINTAINERS
Makefile
mm
modules.builtin
modules.builtin.modinfo
modules-only.symvers
modules.order
Module.symvers
net
OWNERS
README
README.md
resource.img
samples
scripts
security
sound
System.map
tools
usr
virt
vmlinux
vmlinux.o
vmlinux.symvers
zboot.img
```

图 2.2-1 查看 LOGO 图片位置

2.2.3 重新编译内核

更新好 LOGO 图片后，在 SDK 顶层目录执行以下命令重新编译打包内核。

```
./build.sh kernel
```

具体编译手册可详见我司《SDK 使用说明》文档。编译完成，如图 2.2-2 所示。

```
Toolchain for kernel:
/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64
-none-linux-gnu/bin/aarch64-none-linux-gnu-

=====
Start building kernel
=====
+ make -C /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel/ -j33 CROSS_COMPILE=/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-ne-linux-gnu- ARCH=arm64 hzhy_rk3562_mini_defconfig
make: Entering directory '/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel-5.10'
#
# No change to .config
#
make: Leaving directory '/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel-5.10'
RK_KERNEL_CFG: hzhy_rk3562_mini_defconfig
RK_KERNEL_CFG_FRAGMENTS:
+ make -C /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel/ -j33 CROSS_COMPILE=/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-ne-linux-gnu- ARCH=arm64 hzhy_rk3562_mini_defconfig
make: Entering directory '/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel-5.10'
CALL scripts/atomic/check-atomics.sh
CALL scripts/checksyscalls.sh
CHK include/generated/compile.h
Image: resource.img (with hzhy_rk3562_mini.dtb logo.bmp logo_kernel.bmp) is ready
Image: boot.img (with Image resource.img) is ready
Image: zboot.img (with Image.lz4 resource.img) is ready
make: Leaving directory '/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel-5.10'
+ ln -rsf kernel/boot.img /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm64/boot/dts/rockchip/hzhy_rk3562_mini.dts
Running 10-kernel.sh - build kernel succeeded.
```

图 2.2-2 编译完成

2.2.4 烧录并验证 LOGO

编译完成后，重新烧录生成的 boot.img 文件以验证 LOGO 是否更换成功，内核具体烧录步骤详见我司《RK3562_SD 卡启动和系统固化》文档中的“系统固化”部分。

烧录完成后板卡会自动重启，启动时可以看到更换后的启动 LOGO。如图 2.2-3 所示。



图 2.2-3 开机启动 LOGO